

Exact-Output Disclosure for Agent Workflows

Owner approval of exact bytes, consumed once before delivery

Imran Siddique · OPAQUE Systems · October 1, 2026

Technical report, version 1 · Software experiments

Abstract

An answer an agent writes from a confidential record keeps the record's restrictions, even when it reads like a harmless summary. This report describes a release path for that case. An owner signs the exact output bytes together with the recipient, purpose, source scope, inherited labels, policy version and a validity interval. A gate checks the signer's authority for that scope, verifies the signature, and commits a request identifier to a persistent ledger before it calls the one registered delivery adapter. If the acknowledgment is lost, the attempt stays consumed and the outcome is reported as unknown. Nothing is sent twice. We evaluate the implementation in cMCP, which ships in cmcp-runtime 0.6.0 and later. All 46 existing tests pass, and removing either of two controls makes specific tests fail. In 20 rounds of 16 concurrent attempts, every round delivered once. In 10 rounds of 8 competing sender processes, every round produced exactly one recorded receipt. Process kills on both sides of the ledger commit, lock timeouts, a corrupted ledger and a dropped acknowledgment never produced a second receipt; with replay consumption removed, all 9 of those cases fail. Every number was reproduced on October 1, 2026 at the evaluated revision and at current main.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. All results are software results on one Windows host with synthetic payloads and generated Ed25519 keys. The ledger is a local SQLite file, and the recipient is a local process on loopback HTTP. No hardware, cloud resource, networked ledger or independently operated recipient was used. The report does not cover content classification, egress confinement or the semantic safety of the released text. The rerun record, with commands, package versions and outputs, is in experiments/results/rerun-2026-10-01/.

1. The release question

Consider an agent that reads a confidential customer contract and drafts a two-line answer for a partner's service. Before the bytes leave, three questions are open. Does the answer still carry the contract's restrictions? Who has the authority to release it to this partner, for this purpose? And if the network call times out, will a retry send it a second time?

A model saying its answer is public settles none of these. The labels come from the sources the agent read, and they stay attached to anything derived from them. The authority belongs to whoever owns the source. The retry question is about delivery state, which the model never sees.

The mechanism gives each question a concrete answer.

Question	What answers it
Does the output keep its restrictions?	Labels come from the trusted controller, not the prompt, and are part of the signed record.
Who may release it?	A configured authority whose key, source scopes, recipients and purposes are fixed in the gate.
What exactly is released?	The SHA-256 of the exact output bytes is signed, so any change to the bytes invalidates the approval.

Question	What answers it
Can it be sent twice?	The request identifier is committed to a shared ledger before the delivery adapter is called.

The design choice that matters most is the last row. When delivery is uncertain, the gate prefers losing an output to sending it twice. A lost output can be approved again under a fresh identifier once someone has checked what happened. A duplicate disclosure cannot be recalled.

2. Related work

Owner-controlled declassification is not new. Myers and Liskov's decentralized label model lets a principal relax only its own policy on a value, while other owners' policies stay in force [1]. Sabelfeld and Sands classify declassification by what is released, who releases it, where and when [2]. This report applies those ideas to one concrete artifact: the "what" is a byte string fixed by its digest, the "who" is a scoped signer, the "where" is a registered recipient, and the "when" is a signed validity interval plus a single-use identifier.

Transaction authorization in payments already binds approval to the significant transaction data and enforces it on the server, so that altering the data invalidates the approval [3]. The release record here is that pattern applied to agent output, with source scope, inherited labels and delivery-attempt state added.

At-most-once execution is the classic remote procedure call trade-off: when the caller cannot tell whether a call ran, it reports failure instead of repeating the call [4]. The gate makes the same choice for disclosure, where a repeat is the expensive failure.

Rollback of the ledger is the known weak point of any persistent single-use record. Memoir [5] and ROTE [6] address state continuity for protected modules. The gate does not include such a mechanism, and section 6.2 measures what happens without one.

3. Participants and trust

The participants are a trusted controller, an owner-side signer, the release gate, a shared replay ledger, and an operator-registered delivery adapter. The agent supplies candidate content. It does not control the accumulated labels, the recipient registry or the signing key. A recipient identifier maps to a pinned adapter, not to a caller-supplied URL.

The adversary may change any request field, replay an approval, race several callers, put misleading instructions in the prompt, or interrupt delivery after the recipient has the bytes. The argument assumes an uncompromised owner key, authentic controller inputs, a trustworthy clock, a gate policy that does not change during admission, and one protected ledger shared by every participating gate. The registered adapter must authenticate its recipient and send exactly the supplied bytes.

Source discovery and classification happen before the gate. If the controller misses a source or assigns the wrong label, a valid signature does not repair it. An attacker who can write to the process, the recipient registry or the ledger is outside the protection boundary.

4. Protocol

A request carries the output bytes B , workload W , source scope S , labels L , recipient R , purpose U , policy version P and a random 256-bit request identifier I . The approval adds the signer A and a validity interval $[t_0, t_1)$.

The owner signs, with Ed25519 [8], a domain-separated record holding all of these fields and $\text{SHA}_{256}(B)$, serialized with the JSON Canonicalization Scheme [7]. Canonicalization makes the signed bytes deterministic. It

does not make the fields true.

```
trusted controller -> candidate bytes and inherited context
owner signer       -> approval over bytes, context and validity interval
release gate       -> check registry, authority, signature and time
shared ledger      -> commit the request identifier
release gate       -> recheck time after the ledger wait
registered adapter -> one delivery attempt
```

The gate checks in a fixed order. A policy-version mismatch is denied. An unknown recipient, boundary, source scope or label makes the release unavailable. If the recipient is inside an already accepted boundary and every label fits its ceiling, no approval is needed. Otherwise the gate needs an approval from a configured authority whose scopes, recipients and purposes all cover the request. A valid signature from a signer without that scope is denied.

The ledger table has the request identifier as its primary key, and the insert runs in an immediate transaction. Only the caller whose insert succeeds reaches the adapter. Waiting for the ledger lock can use up the rest of the approval's lifetime, so the gate checks the interval again after the insert. A request that fails this second check stays consumed.

The outcome is one of three: no attempt, unknown delivery, or adapter acknowledgment. An acknowledgment means the adapter call returned normally. It is not proof that the recipient stored, processed or protected the information. The returned observation carries a disposition, a reason code, the delivery state and a random event identifier. It does not carry the payload, its digest, the signer or the recipient.

The gate is a separate opt-in library module, `cmcp_runtime.disclosure`. No other cMCP module imports it, at the evaluated revision or at current main. It adds no bypass flag to an ordinary MCP tool call, and the gateway's own sink restrictions still apply on their paths.

5. Properties

Under the stated assumptions, changing any signed field invalidates the approval. A caller cannot swap the approved payload for another and keep a valid approval unless Ed25519 or SHA-256 fails. A permitted disclosure does not relabel the source or anything else derived from it.

For each request identifier in one intact ledger, at most one insert succeeds, so at most one adapter call starts. That is an at-most-once attempt property, not exactly-once delivery. A crash after the commit and before delivery loses the output. A crash after delivery leaves the sender unable to know whether the recipient has it.

Issuing a new identifier for an uncertain attempt is therefore not a safe automatic retry. The application has to keep the link between the business operation and its consumed request, and decide with that information whether to approve again.

6. Evaluation

The evaluated revision is cMCP [2cdb168](#), merged in [cMCP PR 672](#) on September 19, 2026. The same `disclosure.py` ships byte for byte in `cmcp-runtime` 0.6.0 and 0.7.0 on PyPI. The experiments were rerun on October 1, 2026 from a fresh virtual environment with Python 3.12.10, pytest 9.1.1, cryptography 50.0.1 and rfc8785 0.1.4, against the evaluated revision and against current main, [a3f61e1](#). Both runs gave the numbers below, which also match the values first recorded on September 23 and 27, 2026.

6.1 Existing tests and control removals

The unmodified disclosure suite passed all **46 tests**. They cover correct delivery, independent re-serialization of the signed record, substitution of each signed field, scoped authority, early and expired approvals, invalid and failing clocks, storage failure, uncertain delivery, reopening the ledger, and concurrent consumption.

Two controls were removed, one at a time, in a fresh copy of the source.

Change under test	Failing tests	Tests that caught it
Output digest in the signed record replaced by a constant	2 of 46	independent serialization; payload substitution
Replay consumption skipped	4 of 46	expiry recheck after the ledger wait; replay after reopen and failed delivery; shared-ledger concurrency; storage failure

These are targeted checks that the suite depends on these two controls. They are not a mutation score.

6.2 Thread race, lost acknowledgment and rollback

Sixteen attempts with the same approval were submitted in each of 20 rounds, on eight worker threads, each gate opening its own connection to one ledger file. Every round produced one delivery and 15 replay refusals: 20 deliveries from 320 attempts. An adapter that recorded the delivery and then raised a timeout returned `unknown`, and the next attempt was refused as a replay. Copying back a ledger file taken before consumption let the same approval deliver again. That is the storage-trust assumption from section 3, shown directly.

6.3 Process kills, storage faults and a recording receiver

The second experiment runs every sender as a separate operating-system process with its own gate and its own connection to one shared ledger. The adapter posts the bytes over loopback HTTP to a receiver process, which writes each body to an fsync'd log before deciding whether to acknowledge, so receipt and acknowledgment are observed separately. Kill points inside the ledger transaction are injected by wrapping the SQLite connection that the unmodified `ReplayStore.consume` opens; the wrapper ends the process with `os._exit` just before or just after COMMIT. The gate code is not edited.

The hypothesis is that, for one intact ledger, at most one delivery reaches the receiver per request identifier, whatever happens to the sender. A case passes only if it gives exactly the expected row below, with at most one recorded receipt.

Case	First attempt	Ledger rows	Receipts before retry	Retry	Final receipts
Kill before commit	process exits	0	0	acknowledged	1
Kill after commit, before delivery	process exits	1	0	replay refused	0
Kill while the receiver holds the request	process killed	1	1	replay refused	1
Receiver records, then drops the acknowledgment	delivery unknown	1	1	replay refused	1
Commit succeeds, error reported	storage unavailable	1	0	replay refused	0
Ledger file corrupted	storage unavailable	n/a	0	not run	0

Two cases add competition. With the ledger held under an exclusive lock for 8 seconds, longer than SQLite's 5 second wait, all 8 competing processes returned `replay_storage` and nothing was delivered; after the lock

cleared, 8 more attempts produced 1 delivery and 7 replay refusals. With a 3.5 second lock and an approval that expired about 2 seconds after release began, the process that won the insert failed the recheck after the wait, and the other 7 were refused as replays: 0 deliveries, 1 consumed row. That case exercises the recheck described in section 4.

A process race of 10 rounds with 8 sender processes each produced exactly 1 acknowledged delivery, 7 replay refusals and 1 recorded receipt in every round. All 9 cases met the criterion.

The control removes the replay check (`if not self._store.consume(...)` becomes `if False:`) in an isolated copy. None of the 9 cases met the criterion. Every retry after an uncertain delivery produced a second receipt, the locked ledger no longer stopped delivery (16 receipts across the two phases), and the process race delivered 8 times in every round.

Two rows show the design choice directly. A kill after the commit, and a commit whose success is reported as an error, both lose the output rather than risk sending it twice. Only the receiver's log tells "received but not acknowledged" apart from "never received". The sender sees the same `delivery_unknown` in both.

7. Limits

Rollback protection, egress confinement, trusted classification and distributed replacement of gate policy are outside the gate. Restoring an earlier ledger re-enables an approval, as section 6.2 shows, so a deployment that needs rollback resistance has to keep the ledger on storage with state continuity of the kind Memoir or ROTE provide. The gate has no delivery watchdog; a blocking adapter needs its own timeout and resource limit. A low-entropy output can be guessed from its digest, so approval records belong in private storage.

All runs used one Windows host and a local SQLite file. A ledger shared across hosts over a network, and a receiver operated by a different party, are untested. Suite duration is not a latency benchmark, and the experiments measure no throughput.

8. Conclusion

An approved byte string can be tied to a scoped, persistent, single-attempt release decision, and an uncertain delivery is reported as uncertain instead of being repeated. A team can check this for itself: the 46 tests, the two control removals, and both experiments with their control rerun with one command against any cMCP clone.

The same mechanism appears as one component of the broader report *Confidentiality Across Agent Handoffs* (OPAQUE Systems technical report, version 2, September 30, 2026, DOI [10.5281/zenodo.23074670](https://doi.org/10.5281/zenodo.23074670)), which composes it with provisioning, confinement and delegated response authentication. This report is the focused study with its own pinned evaluation; it does not change the results of that one.

Appendix A. Reproduction

Item	Value
Evaluated revision	cMCP 2cdb168ce52020406ff0ef6cbc34447f0ea55aee
Current main rerun	cMCP a3f61e1a4d866bcd4fd2a149fff4da34b7e68597, which requires agent-manifest 0.14.0 or later (run with 0.15.0)
Released package	cmcp-runtime 0.6.0 and 0.7.0 contain the evaluated <code>disclosure.py</code> unchanged
Environment	Windows 11, Python 3.12.10, pytest 9.1.1, pytest-asyncio 1.4.0, cryptography 50.0.1, rfc8785 0.1.4
Command	<code>python -B experiments/rerun.py --repo <cmcp clone> --output <new directory> [--revision <commit>]</code>
Records	experiments/results/rerun-2026-10-01/evaluated-2cdb168/ and main-a3f61e1/, each with summary.json, JUnit XML, logs, experiment JSON and SHA256SUMS.json

The runner exports the chosen revision with `git archive`, runs the suite, the two control removals, `concurrency.py`, and `delivery_faults.py` with its control, then compares every count with the values recorded when the report was written. Local paths are replaced by placeholders before anything is written.

Implementation at the evaluated revision: [release gate](#), [release contract](#), [test suite](#).

References

- [1] A. C. Myers and B. Liskov. [Complete, Safe Information Flow with Decentralized Labels](#). IEEE Symposium on Security and Privacy, 1998.
- [2] A. Sabelfeld and D. Sands. [Declassification: Dimensions and principles](#). Journal of Computer Security 17(5):517 to 548, 2009.
- [3] OWASP. [Transaction Authorization Cheat Sheet](#). OWASP Cheat Sheet Series.
- [4] A. D. Birrell and B. J. Nelson. [Implementing Remote Procedure Calls](#). ACM Transactions on Computer Systems 2(1):39 to 59, 1984.
- [5] B. Parno, J. R. Lorch, J. R. Douceur, J. Mickens and J. M. McCune. [Memoir: Practical State Continuity for Protected Modules](#). IEEE Symposium on Security and Privacy, 2011.
- [6] S. Matetic, M. Ahmed, K. Kostiainen, A. Dhar, D. Sommer, A. Gervais, A. Juels and S. Čapkun. [ROTE: Rollback Protection for Trusted Execution](#). USENIX Security Symposium, 2017.
- [7] A. Rundgren, B. Jordan and S. Erdtman. [JSON Canonicalization Scheme \(JCS\)](#). RFC 8785, June 2020.
- [8] S. Josefsson and I. Liusvaara. [Edwards-Curve Digital Signature Algorithm \(EdDSA\)](#). RFC 8032, January 2017.